JAVA’YA GİRİŞ

Temelden başlayarak Java kodunu okuma, yazma ve çalıştırma rehberi

12 temel konu
Her konu için sade anlatım + ayrı kod örneği sayfası

Hedef kitle: Java ile ilk kez tanışan öğrenciler

Kaynak yaklaşımı: Oracle Java Tutorials ve güncel Java öğrenme kaynaklarının temel kavramları esas alınmıştır.

İçindekiler

1. Java nedir ve nerelerde kullanılır?
2. Java’nın çalışma mantığı
3. JDK, JRE ve JVM
4. Java syntax kuralları
5. Değişkenler ve veri tipleri
6. Operatörler
7. Scanner ile kullanıcıdan veri alma
8. Koşul ifadeleri
9. Döngüler
10. Metotlar
11. Diziler
12. Temel hata türleri

> Bu doküman nasıl kullanılmalı?
> Önce anlatım sayfasını okuyun. Ardından hemen arkasındaki kod sayfasını çalıştırın ve küçük değişiklikler yaparak sonucu gözlemleyin. Kod görmek kadar kodu değiştirip sonucu tahmin etmek de önemlidir.

1. Java nedir ve nerelerde kullanılır?

Java, genel amaçlı ve güçlü tip sistemine sahip bir programlama dilidir. İlk sürümü 1995 yılında yayımlandı. Java sadece bir dil değildir; aynı zamanda kodun farklı işletim sistemlerinde çalışmasını kolaylaştıran geniş bir çalışma ortamına ve ekosisteme sahiptir.

Java'nın en çok bilinen fikirlerinden biri "Write Once, Run Anywhere" yaklaşımıdır. Buradaki amaç, Java kaynak kodunu doğrudan her işletim sistemine özel ayrı ayrı yazmak yerine, Java Virtual Machine (JVM) bulunan farklı ortamlarda çalıştırabilmektir.

Java bugün özellikle büyük ve uzun ömürlü sistemlerde sık görülür. Bankacılık, finans, e-ticaret, telekomünikasyon, kamu uygulamaları, kurumsal backend sistemleri ve mikroservis mimarileri Java'nın yaygın olduğu alanlardır.

Java ile yapılabilecek başlıca işler

- Web uygulamalarının backend servislerini geliştirmek.
- REST API ve mikroservisler oluşturmak.
- Kurumsal masaüstü uygulamaları geliştirmek.
- Veritabanı ile çalışan iş uygulamaları hazırlamak.
- Mesajlaşma, ödeme, sipariş, stok gibi yüksek trafikli servisler geliştirmek.
- Android ekosistemindeki mevcut Java kod tabanlarını geliştirmek ve bakımını yapmak.

> Önemli
> Java eski bir dil olabilir; ancak "eski" olması "kullanılmıyor" anlamına gelmez. Uzun yıllardır geliştirilmesi, büyük ekosistemi ve geriye dönük uyumluluğa verilen önem, kurumsal dünyada güçlü kalmasının nedenlerindendir.

Kod Örneği 1 - İlk Java Programı

Bir Java programının en basit haliyle ekrana nasıl yazı yazdırdığını görelim.

```
public class MerhabaJava {
    public static void main(String[] args) {
        System.out.println("Merhaba Java!");
    }
}
```

Kodu okurken dikkat et

- public class MerhabaJava: MerhabaJava adında bir sınıf tanımlar.
- main metodu programın başlangıç noktasıdır.
- System.out.println(...) ekrana metin yazdırır.
- Dosya adı public sınıf adıyla aynı olmalıdır: MerhabaJava.java

2. Java’nın çalışma mantığı

Java kaynak kodu doğrudan işlemcinin anlayacağı makine kodu olarak yazılmaz. Önce Java derleyicisi tarafından bytecode adı verilen ara bir forma dönüştürülür. Bu bytecode daha sonra JVM tarafından çalıştırılır.

Temel akış

Derleme ve çalıştırma akışı
Kaynak kod (.java) → javac derleyicisi → Bytecode (.class) → JVM → Programın çalışması

Bu yapı neden önemlidir?

- Kaynak kod ile işletim sistemi arasına JVM katmanı girer.
- Aynı .class dosyası, uygun JVM bulunan farklı sistemlerde çalıştırılabilir.
- JVM çalışma sırasında bellek yönetimi, güvenlik kontrolleri ve performans optimizasyonları gibi görevler üstlenir.

Java’da derleme ve çalıştırma iki ayrı kavramdır. Derlemek, kaynak kodu bytecode’a çevirmektir. Çalıştırmak ise oluşan bytecode’u JVM üzerinde yürütmektir.

Kod Örneği 2 - Derleme ve Çalıştırma

Komut satırında basit bir Java dosyasını derleyip çalıştırma mantığı:

```
// 1) Dosya: MerhabaJava.java
public class MerhabaJava {
    public static void main(String[] args) {
        System.out.println("Java çalışıyor!");
    }
}

// 2) Derleme komutu
javac MerhabaJava.java

// 3) Oluşan dosya
MerhabaJava.class

// 4) Çalıştırma komutu
java MerhabaJava
```

Kodu okurken dikkat et

- javac kaynak kodu derler.
- Derleme başarılıysa .class dosyası oluşur.
- java komutu sınıfı JVM üzerinde çalıştırır.
- Çalıştırırken .class uzantısı yazılmaz.

3. JDK, JRE ve JVM

Bu üç kavram başlangıçta birbirine çok benzer görünür; ancak görevleri farklıdır.

JVM - Java Virtual Machine

Bytecode'u çalıştıran sanal makinedir. İşletim sistemi ile Java programı arasında çalışma katmanı oluşturur. JVM; bytecode yürütme, bellek yönetimi ve çöp toplama gibi görevlerde rol alır.

JRE - Java Runtime Environment

Tarihsel olarak Java uygulamalarını çalıştırmak için gereken JVM ve çalışma kütüphanelerini ifade eder. Modern Java dağıtımlarında ayrı bir son kullanıcı JRE paketi eskisi kadar merkezi değildir; ancak kavram, Java'nın çalışma ortamını açıklamak için hâlâ önemlidir.

JDK - Java Development Kit

Java geliştirmek için gereken araçların bütünüdür. Derleyici (javac), java çalıştırıcısı ve başka geliştirme araçlarını içerir. Kod yazıp derleyeceksiniz temel olarak JDK gerekir.

> Kısaca
> JDK = geliştirmek için
> JRE = çalıştırma ortamı kavramı
> JVM = bytecode'u çalıştıran sanal makine

Kod Örneği 3 - Java Kurulumunu Kontrol Etme

JDK kurulduktan sonra terminal veya komut isteminde aşağıdaki komutlarla kontrol yapılabilir.

```
java -version

javac -version
```

Kodu okurken dikkat et

- java -version çalışan Java sürümünü gösterir.
- javac -version Java derleyicisinin sürümünü gösterir.
- javac komutu bulunamıyorsa JDK kurulumu veya PATH ayarı kontrol edilmelidir.

4. Java syntax kuralları

Syntax, bir programlama dilinde kodun hangi kurallara göre yazılacağını ifade eder. Türkçede cümle kuralları olduğu gibi Java’da da derleyicinin anlayacağı kesin yazım kuralları vardır.

Temel kurallar

- Java büyük-küçük harfe duyarlıdır. isim ve Isim farklı adlardır.
- Birçok ifade noktalı virgül (;) ile biter.
- Kod blokları süslü parantezler { } ile gösterilir.
- Tek satırlık yorum // ile, çok satırlık yorum /* ... */ ile yazılır.
- Sınıf adlarında genellikle PascalCase, değişken ve metot adlarında camelCase kullanılır.
- Anahtar kelimeler (class, public, int, if gibi) değişken adı olarak kullanılamaz.

Java’da okunabilir kod önemlidir. Girintiler derleyici için zorunlu olmasa da insanlar için kodun yapısını anlamayı çok kolaylaştırır.

```
Örnek adlandırma
Sınıf: OgrenciKayit
Değişken: ogrenciAdi
Metot: notHesapla()
```

Kod Örneği 4 - Syntax Kuralları

Aynı program içinde sınıf, değişken, yorum, kod bloğu ve noktalı virgül kullanımını görelim.

```
public class SyntaxOrnegi {
    public static void main(String[] args) {
        // Bu bir yorum satırıdır.
        int yas = 25;
        String isim = "Ayşe";

        System.out.println(isim);
        System.out.println(yas);
    }
}
```

Kodu okurken dikkat et

- yas ve isim değişken adları camelCase yaklaşımına uygundur.
- Her atama ve println ifadesi ; ile biter.
- main metodunun içeriği { } arasında yer alır.
- String büyük S ile yazılır; string yazarsanız aynı şey değildir.

5. Değişkenler ve veri tipleri

Değişken, program çalışırken bir değeri saklamak için kullanılan isimlendirilmiş alandır. Java güçlü tipli bir dildir; yani bir değişken tanımlanırken hangi türde veri tutacağı belirtilir.

Temel primitive veri tipleri

Tip	Ne tutar?	Örnek
byte	Küçük tam sayılar	byte seviye = 5;
short	Tam sayılar	short yil = 2026;
int	En yaygın tam sayı	int yas = 29;
long	Büyük tam sayılar	long nufus = 90000000L;
float	Ondalıklı sayı	float oran = 1.5f;
double	Daha hassas ondalık	double fiyat = 199.99;
char	Tek karakter	char not = 'A';
boolean	true / false	boolean aktif = true;

String primitive değildir; metinleri temsil eden bir sınıftır. Buna rağmen başlangıç seviyesinde en sık kullanılan veri tiplerinden biridir.

Değişken tanımlama mantığı

Şablon
veriTipi değişkenAdi = değer;
Örnek: int yas = 29;

Kod Örneği 5 - Değişkenler ve Veri Tipleri

Farklı türlerde verileri saklayıp ekrana yazdıralım.

```
public class VeriTipleri {
    public static void main(String[] args) {
        String ad = "Deniz";
        int yas = 22;
        double boy = 1.68;
        char grup = 'A';
        boolean ogrenciMi = true;

        System.out.println(ad);
        System.out.println(yas);
        System.out.println(boy);
        System.out.println(grup);
        System.out.println(ogrenciMi);
    }
}
```

Kodu okurken dikkat et

- String değerler çift tırnak içinde yazılır.
- char tek karakter tutar ve tek tırnak kullanır.
- boolean yalnızca true veya false olabilir.
- double ondalıklı sayılar için uygundur.

6. Operatörler

Operatörler, değerler ve değişkenler üzerinde işlem yapmamızı sağlayan sembollerdir.

Aritmetik operatörler

- + toplama
- - çıkarma
- * çarpma
- / bölme
- % mod - bölümden kalanı bulur

Karşılaştırma operatörleri

- == eşit mi?
- != eşit değil mi?
- > büyük mü?
- < küçük mü?
- >= büyük veya eşit mi?
- <= küçük veya eşit mi?

Mantıksal operatörler

- && ve
- || veya
- ! değil

Atama ve artırma

- = değer atar.
- +=, -= gibi operatörler mevcut değerin üzerine işlem yapar.
- ++ değeri 1 artırır, -- değeri 1 azaltır.

> Dikkat
> = atama yapar; == karşılaştırma yapar. Başlangıç seviyesinde en sık karıştırılan noktalardan biridir.

Kod Örneği 6 - Operatörler

Aritmetik ve karşılaştırma operatörlerini aynı örnekte görelim.

```
public class Operatorler {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;

        System.out.println(a + b); // 13
        System.out.println(a - b); // 7
        System.out.println(a * b); // 30
        System.out.println(a / b); // 3
        System.out.println(a % b); // 1

        System.out.println(a > b);  // true
        System.out.println(a == b); // false
    }
}
```

Kodu okurken dikkat et

- int ile int bölündüğünde sonuç yine int olur; 10 / 3 sonucu 3’tür.
- % operatörü kalan değeri verir.
- Karşılaştırma işlemlerinin sonucu boolean yani true/false olur.

7. Scanner ile kullanıcıdan veri alma

Programların yalnızca sabit değerlerle değil, kullanıcıdan gelen verilerle de çalışması gerekir. Konsoldan veri almak için başlangıç seviyesinde en yaygın araçlardan biri Scanner sınıfıdır.

Scanner kullanmanın temel adımları

- java.util.Scanner sınıfını import etmek.
- System.in üzerinden çalışan bir Scanner nesnesi oluşturmak.
- nextLine(), nextInt(), nextDouble() gibi metotlarla veriyi okumak.
- İşimiz bittiğinde scanner.close() ile kaynağı kapatmak.

Sık kullanılan metotlar

- nextLine(): Satır şeklinde metin okur.
- nextInt(): int değer okur.
- nextDouble(): double değer okur.

> Sık karşılaşılan durum
> nextInt() sonrasında nextLine() kullanıldığında satır sonu karakteri nedeniyle beklenmedik sonuç görülebilir. Başlangıç seviyesinde bunu fark etmek önemlidir; gerekli durumda araya ekstra nextLine() eklenir.

Kod Örneği 7 - Scanner

Kullanıcıdan ad ve yaş bilgisini alıp ekrana yazdıralım.

```
import java.util.Scanner;

public class KullaniciGirdisi {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Adınız: ");
        String ad = scanner.nextLine();

        System.out.print("Yaşınız: ");
        int yas = scanner.nextInt();

        System.out.println("Merhaba " + ad);
        System.out.println("Yaşınız: " + yas);

        scanner.close();
    }
}
```

Kodu okurken dikkat et

- Scanner sınıfı java.util paketinden import edilir.
- System.in klavyeden gelen girdiyi temsil eder.
- + operatörü burada metin birleştirmek için kullanılır.
- scanner.close() kullanılan kaynağı kapatır.

8. Koşul ifadeleri

Koşul ifadeleri, programın bir duruma göre farklı davranmasını sağlar. Örneğin kullanıcı 18 yaşından büyükse bir işlem yapmak, değilse başka mesaj göstermek koşullu çalışmadır.

if

Koşul true olduğunda blok çalışır.

if - else

Koşul true ise bir blok, false ise başka blok çalışır.

else if

Birden fazla koşulu sırayla kontrol etmek için kullanılır.

switch

Tek bir değerin farklı olası durumlarına göre seçim yapmak için kullanılabilir. Özellikle menü seçimi veya durum kodları gibi senaryolarda okunabilir olabilir.

> Mantık
> Koşulların sonucu boolean olmalıdır: yani sonuç true veya false olmalıdır.

Kod Örneği 8 - Koşul İfadeleri

Bir öğrencinin notuna göre basit sonuç üretelim.

```
public class Kosullar {
    public static void main(String[] args) {
        int not = 72;

        if (not >= 85) {
            System.out.println("Pekiyi");
        } else if (not >= 70) {
            System.out.println("İyi");
        } else if (not >= 50) {
            System.out.println("Geçti");
        } else {
            System.out.println("Kaldı");
        }
    }
}
```

Kodu okurken dikkat et

- Koşullar yukarıdan aşağı doğru değerlendirilir.
- İlk true olan bloğa girildikten sonra diğer else-if blokları atlanır.
- Sınır değerlerini özellikle test edin: 49, 50, 69, 70, 84, 85.

9. Döngüler

Döngüler, aynı veya benzer işlemi tekrar tekrar yazmak yerine belirli bir kurala göre tekrarlamamızı sağlar.

for döngüsü

Tekrar sayısı belli olduğunda sık kullanılır. Sayaç başlangıcı, devam koşulu ve sayaç değişimi tek satırda görülebilir.

while döngüsü

Koşul true olduğu sürece çalışır. Kaç kere döneceğinin başlangıçta bilinmediği durumlarda uygundur.

do-while döngüsü

Koşulu sonda kontrol eder. Bu nedenle gövde en az bir kez çalışır.

break ve continue

- break döngüyü tamamen sonlandırır.
- continue mevcut turu atlar ve sonraki tura geçer.

> Dikkat
> Döngü koşulu hiçbir zaman false olmayacak şekilde yazılırsa sonsuz döngü oluşabilir.

Kod Örneği 9 - Döngüler

1’den 5’e kadar sayıları for döngüsüyle yazdıralım.

```
public class Donguler {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

Kodu okurken dikkat et

- int i = 1 başlangıç değeridir.
- i <= 5 devam koşuludur.
- i++ her tur sonunda i değerini 1 artırır.
- Koşul false olduğunda döngü biter.

10. Metotlar

Metot, belirli bir işi yapan ve gerektiğinde tekrar çağrılabilen kod bloğudur. Metotlar kod tekrarını azaltır ve programı küçük, anlaşılır parçalara ayırmayı kolaylaştırır.

Bir metodun temel parçaları

- Erişim belirleyici: public, private gibi.
- static gibi ek belirleyiciler olabilir.
- Dönüş tipi: int, String, boolean veya değer döndürmüyorsa void.
- Metot adı.
- Parametreler.
- Metot gövdesi.

Parametre ve return

Parametre, metoda dışarıdan gönderilen bilgidir. return ise metodun ürettiği değeri çağıran yere geri gönderir.

> Amaç
> Uzun bir programı tek parça yazmak yerine, “hesapla”, “yazdir”, “kontrolEt” gibi küçük görevleri metotlara ayırmak okunabilirliği artırır.

Kod Örneği 10 - Metotlar

İki sayıyı toplayan ve sonucu geri döndüren basit bir metot yazalım.

```
public class MetotOrnegi {
    public static int topla(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int sonuc = topla(4, 6);
        System.out.println(sonuc);
    }
}
```

Kodu okurken dikkat et

- topla metodu iki int parametre alır.
- Dönüş tipi int olduğu için return ile int değer döndürmelidir.
- topla(4, 6) çağrısı sonucunda 10 elde edilir.
- Bu aşamada static ayrıntısını derinleştirmek yerine kullanım mantığını anlamak yeterlidir.

11. Diziler

Dizi (array), aynı tipte birden fazla değeri tek bir yapı altında tutmamızı sağlar. Dizinin eleman sayısı oluşturulduğu anda belirlenir ve klasik Java dizilerinde sonradan değişmez.

Index mantığı

Java dizilerinde ilk elemanın index değeri 0’dır. Beş elemanlı bir dizinin indexleri 0, 1, 2, 3 ve 4 olur.

Dizi oluşturma

```
İki yaygın yol
int[] sayilar = new int[3];
veya
int[] sayilar = {10, 20, 30};
```

Dizilerde dikkat edilmesi gerekenler

- Dizi yalnızca belirtilen tipte değerler tutar.
- length alanı dizi uzunluğunu verir.
- Olmayan bir indexe erişmek çalışma zamanında hata üretir.

Collections konusu bu eğitim akışında ayrı bir başlık olduğu için ArrayList gibi dinamik koleksiyonlar burada ele alınmaz.

Kod Örneği 11 - Diziler

Bir isim dizisini for döngüsü ile ekrana yazdıralım.

```
public class DiziOrnegi {
    public static void main(String[] args) {
        String[] isimler = {"Ayşe", "Can", "Deniz"};

        for (int i = 0; i < isimler.length; i++) {
            System.out.println(isimler[i]);
        }
    }
}
```

Kodu okurken dikkat et

- İlk eleman isimler[0] ile alınır.
- isimler.length dizinin eleman sayısını verir.
- i < isimler.length koşulu son geçerli indexin dışına çıkmayı önler.

12. Temel hata türleri

Programlama öğrenirken hata görmek normaldir. Önemli olan hatayı türüne göre okuyup nerede arama yapacağımızı bilmektir.

1) Syntax / Compile-time hataları

Kod Java'nın yazım kurallarına uymuyorsa veya derleyici kodu geçerli şekilde derleyemiyorsa ortaya çıkar. Örneğin ; unutmak veya tanımlanmamış değişken kullanmak.

2) Runtime hataları

Kod derlenir; ancak program çalışırken hata oluşur. Örneğin sıfıra bölme, geçersiz dizi indexi veya null bir nesneye erişim.

3) Mantıksal hatalar

Program hata vermeden çalışır fakat yanlış sonuç üretir. Derleyici bunu her zaman yakalayamaz. Örneğin ortalama hesabında yanlış formül kullanmak.

Hata mesajını okuma alışkanlığı

- Hangi dosya ve satırda hata olduğunu kontrol edin.
- Hata sınıfının veya mesajının adını okuyun.
- Hatanın oluştuğu satırdan önceki değerleri kontrol edin.
- Birden fazla hata varsa önce en üstteki temel hatayı çözmek çoğu zaman diğerlerini de azaltır.

> Eğitim önerisi
> Öğrenciye yalnızca doğru kodu değil, bilinçli olarak hatalı kodu da çalıştırın. Hata mesajı okumak Java öğrenmenin temel becerilerinden biridir.

Kod Örneği 12 - Üç Farklı Hata

Aşağıdaki parçalar üç temel hata türünü göstermektedir.

```
// 1) Compile-time hata
int sayi = 10
// Noktalı virgül eksik.

// 2) Runtime hata
int sonuc = 10 / 0;
// Derlenebilir, çalışırken ArithmeticException oluşur.

// 3) Mantıksal hata
int a = 10;
int b = 20;
int ortalama = a + b / 2;
// Amaç (a + b) / 2 ise sonuç yanlıştır.
```

Kodu okurken dikkat et

- Compile-time hatasında program çalışmadan önce derleme başarısız olur.
- Runtime hatasında program çalışmaya başladıktan sonra sorun çıkar.
- Mantıksal hatada program çalışabilir ancak beklenen sonucu vermez.
- Parantez önceliği gibi küçük ayrıntılar mantıksal hata yaratabilir.

Kısa Tekrar

Bu bölüm tamamlandığında öğrencinin şu becerilere sahip olması beklenir:

- Java’nın ne olduğunu ve nerelerde kullanıldığını açıklayabilmek.
- Kaynak kod → bytecode → JVM çalışma akışını anlatabilmek.
- JDK, JRE ve JVM kavramlarını birbirinden ayırabilmek.
- Temel Java syntax kurallarını kullanabilmek.
- Değişken, veri tipi ve operatörlerle basit işlemler yapabilmek.
- Scanner ile konsoldan veri alabilmek.
- if/switch ile karar verebilmek ve döngülerle tekrar yapabilmek.
- Basit metot ve diziler yazabilmek.
- Temel hata türlerini ayırt edip hata mesajı okumaya başlayabilmek.

> Sonraki adım
> Bu temel üzerine Eclipse kullanımı, Swing bileşenleri, dosya işlemleri, nesne yönelimli programlama, Collections ve veritabanı bağlantıları daha rahat oturacaktır.

Kaynakça ve Notlar

Bu dokümanın temel kavramsal çerçevesi aşağıdaki resmi Java kaynaklarıyla uyumlu hazırlanmıştır:

- Oracle Java Tutorials - Learning the Java Language: https://docs.oracle.com/javase/tutorial/java/
- Oracle Java Tutorials - Language Basics: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/
- Oracle Java Tutorials - Variables: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/variables.html
- Oracle Java Tutorials - Operators: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html
- Oracle Java Tutorials - Control Flow Statements: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html
- Oracle Java Tutorials - Arrays: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

Not: Oracle’ın klasik Java Tutorials içeriğinin önemli bir bölümü JDK 8 döneminde hazırlanmıştır. Temel syntax, değişkenler, operatörler, kontrol akışı ve diziler gibi başlangıç kavramları için hâlâ yararlı bir kaynaktır. Daha güncel Java öğrenme içerikleri için dev.java da kullanılabilir.

Kod örnekleri eğitim amacıyla sadeleştirilmiştir. Nesne yönelimli programlama, Collections ve ileri seviye dil özellikleri bu dokümanın kapsamı dışında bırakılmıştır.